Binary Suffix Learning in Non-Binary Bases

Project Outline (COMPSCI 590 Final Project)

---

1. Motivation and Research Question

Charton & Narayanan (2025) train small transformers to predict the "long Collatz step" — a complex arithmetic function — with inputs and outputs encoded in various bases (2 through 57). They observe a striking phenomenon: **all models, regardless of encoding base, learn to classify inputs by their residues modulo powers of 2**. That is, models internally learn to detect the binary suffix of their input, even when the input is presented in, say, base 27 or base 43.

The authors call this "a mystery," noting that the standard algorithm for computing $n \bmod 2^p$ from a base-B representation requires a weighted modular sum over all digits — a task known to be hard for transformers (Saxena et al., 2025).

This project isolates and studies that mystery. We strip away the Collatz arithmetic entirely and ask:

> **Can a small transformer learn the function $f(n) = n \bmod 2^p$, given n encoded as a digit sequence in base B? How does accuracy depend on B and p? What representations does the model learn internally?**

This is a clean, self-contained question. The task is mathematically well-defined, the data is synthetic (unlimited, no contamination concerns), and the expected difficulty gradient — easy when B is a power of 2, hard when B is odd — gives us a structured landscape to explore.

---

2. Mathematical Background

You already know this, but writing it down precisely will help you design experiments and interpret results.

2.1 The task

Fix a base $B \geq 2$ and a modulus 2^p for some $p \geq 1$. The input is the base-B representation of a positive integer n, given as a sequence of tokens $(d_L, d_{L-1}, \ldots, d_1, d_0)$ where $n = \sum_{i=0}^{L} d_i B^i$. The target output is $n \bmod 2^p$, encoded as a sequence of tokens in the same base B (or optionally in base 2 — see Section 4.3).

2.2 When the task is trivial

If $B = 2^k$ for some k, then $n \bmod 2^p$ depends only on the last $\lceil p/k \rceil$ tokens. A model with even

a single attention layer can learn to copy those tokens. This is the "easy" baseline.

2.3 When the task is hard

If $\gcd(B, 2) = 1$ (i.e., B is odd), then the residue $n \bmod 2^p$ depends on *all* digits, via

$$n \bmod 2^p = \left(\sum_{i=0}^{L} d_i \cdot B^i \right) \bmod 2^p.$$

This is a weighted modular sum. The weights $B^i \bmod 2^p$ cycle with some period dividing $\phi(2^p) = 2^{p-1}$ (for $p \geq 3$). The model must learn to compute this sum, which prior work suggests is difficult for transformers.

2.4 The intermediate case

If B is even but not a power of 2 — say $B = 2^a \cdot m$ with m odd and $a \geq 1$ — then the last $\lceil p/a \rceil$ digits determine $n \bmod 2^p$ only when $p \leq a$. For larger p, partial information comes from the suffix, but the full residue still requires a nontrivial computation. This creates a smooth difficulty gradient to study.

---

3. Experimental Setup

3.1 Model architecture

Use a **small encoder-decoder transformer**, matching the Charton & Narayanan setup as closely as is practical:

- Encoder: 4 layers, embedding dim 256 or 512, 8 attention heads
- Decoder: 1 layer, same embedding dim, 8 heads, cross-attention to encoder
- Vocabulary: B digit tokens + special tokens (BOS, EOS, PAD)

If you want to simplify, an **encoder-only** model with a classification head also works for this task (since the output space is finite: 2^p possible residues). This is arguably cleaner and easier to probe:

- Encoder: 4 layers, dim 256, 8 heads
- Classification head: linear layer mapping the CLS token (or mean-pooled) representation to 2^p classes

Implementation. Use PyTorch. You do *not* need HuggingFace or any heavyweight library. A from-scratch transformer for this task is ~200 lines. If you prefer a starting point, use

`nn.TransformerEncoder` from PyTorch's standard library.

3.2 Data generation

This is the easiest part. For each training example:

1. Sample n uniformly at random from $[1, N_{\max}]$ (e.g., $N_{\max} = 10^8$ or 10^{10}).
2. Compute the base-B digit sequence of n.
3. Compute $t = n \bmod 2^p$.
4. Encode t as a base-B digit sequence (for seq2seq), or as a class label in $\{0, 1, \ldots, 2^p - 1\}$ (for classification).

Generate data on-the-fly during training (no need to store datasets). Use Python's built-in integer arithmetic — it handles arbitrary precision natively.

Sequence length. For $N_{\max} = 10^{10}$ in base 3, the longest input is $\lceil \log_3(10^{10}) \rceil = 21$ tokens. In base 57, it's 6 tokens. Pad all sequences to a fixed maximum length per base.

3.3 Training details

- **Optimizer:** Adam, learning rate $\sim 3 \times 10^{-4}$ (tune if needed).
- **Loss:** Cross-entropy (either token-level for seq2seq, or over 2^p classes for classification).
- **Batch size:** 512–2048 (these are tiny sequences; you can fit large batches).
- **Training duration:** Monitor accuracy on a held-out validation set. Expect to need 10^6–10^7 examples depending on task difficulty. On a single GPU, each experiment should take minutes to a few hours.
- **Evaluation:** Exact-match accuracy on a fresh test set of 10,000–100,000 examples (no overlap with training since data is generated randomly from a huge space).

3.4 Experimental grid

The core experiment is a sweep over:

- **Bases B:** 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 16, 24, 27, 32. (This covers powers of 2, odd primes, and mixed cases.)
- **Modulus p:** 1, 2, 3, 4, 5, 6, 7, 8. (Start small; $p = 1$ is just parity detection.)

That's $15 \times 8 = 120$ experiments, each small. If compute is limited, prioritize a smaller grid: $B \in \{2, 3, 5, 8, 10, 16, 27\}$ and $p \in \{1, 2, 4, 6, 8\}$ (35 experiments).

---

4. Hypotheses and Analysis Plan

4.1 Primary hypothesis: accuracy depends on $\nu_2(B)$

Let $\nu_2(B)$ denote the 2-adic valuation of B (the largest k such that $2^k | B$).

Hypothesis: For fixed p, model accuracy is primarily determined by $\nu_2(B)$:

- If $\nu_2(B) \geq p$, the task is trivially solvable from the last token. Accuracy $\to$ 100% quickly.
- If $0 < \nu_2(B) < p$, accuracy decreases as $p/\nu_2(B)$ grows.
- If $\nu_2(B) = 0$ (odd base), accuracy is low and drops sharply with p.

How to test: Plot accuracy as a function of p for each base, grouped by $\nu_2(B)$. If the hypothesis is correct, the curves should cluster by 2-adic valuation. Also plot accuracy vs. the "effective difficulty" $\lceil p/\max(\nu_2(B), 1) \rceil$ and check whether curves collapse.

4.2 Secondary hypothesis: step-like learning curves

Hypothesis: Models learning odd-base residues exhibit the same step-like learning pattern observed in the Collatz paper: accuracy jumps in discrete increments corresponding to learning one additional bit of the binary residue.

For the parity task ($p = 1$), there is only one step. For $p = 4$, you'd expect accuracy to plateau near $1/2$, then $3/4$, then $7/8$, then jump to $15/16 \approx 93.75\%$ or 100%, corresponding to the model learning successive bits of $n \bmod 16$.

How to test: Log accuracy at fine-grained intervals during training. Look for plateau–jump patterns. Compare the observed plateaus to the predicted values $1 - 1/2^j$ for $j = 1, \ldots, p$.

4.3 Probing internal representations (the "interesting" part)

This is where the project connects to the deeper question of *how* odd-base models solve this task.

Linear probe for binary features. After training a model on base B with target $n \bmod 2^p$:

1. Freeze the trained encoder.
2. For each bit position $j \in \{0, 1, \ldots, p-1\}$, train a linear classifier (logistic regression) on the encoder's hidden states to predict the j-th bit of $n \bmod 2^p$.
3. Measure probe accuracy as a function of layer depth.

What to look for:

- In power-of-2 bases, probe accuracy should be near 100% even at early layers (the information is trivially available in the input tokens).

- In odd bases, if the model has learned an internal binary encoding, probe accuracy should increase with depth, reaching high values at the final layer. If probes succeed at intermediate layers, that tells you *where* the base conversion happens.
- If probes fail (low accuracy at all layers), the model is using a non-binary internal representation — which would itself be an interesting finding.

Attention pattern analysis. For odd-base models that succeed:

- Visualize attention weights. Does the model attend to all input tokens (consistent with a weighted sum) or only to a suffix (inconsistent with the mathematical requirement for odd bases)?
- Compare attention patterns across p values: does the model attend to more tokens as p grows?

4.4 Ablation: output encoding

Run a variant where the *input* is in base B but the *output* (the residue $n \bmod 2^p$) is always encoded in base 2 (as a p-bit binary string). This tests whether the difficulty lies in the input side (reading the base-B representation) or the output side (writing the answer in base B). If output-in-base-2 dramatically improves accuracy for odd-B inputs, that points to the output encoding as a bottleneck.

---

5. Practical Tips for Implementation

5.1 Start with parity ($p = 1$)

Parity detection — predicting $n \bmod 2$ — is the simplest version of the task. In an odd base, this requires computing a weighted sum of all digits mod 2. Get this working first. If your model can't learn parity in base 3, something is wrong with your setup (or it's a real negative result worth reporting).

5.2 Sanity checks

- **Base 2, any p:** The model should reach 100% accuracy almost immediately (the answer is just the last p tokens of the input). If it doesn't, your tokenization or data pipeline has a bug.
- **Any base, $p = 0$ (if you include it):** Trivial (the answer is always 0). Not useful, but helps check that training runs.
- **Base 4, $p = 2$:** The answer is the last token. Should be trivial.
- **Base 4, $p = 3$:** The answer depends on the last two tokens. Should still be easy but requires combining information across positions.

5.3 Code structure suggestion

```
project/
├── data.py          # Data generation: sample n, compute digits, compute target
├── model.py         # Transformer architecture (encoder-only or encoder-decoder)
├── train.py         # Training loop, logging, checkpoint saving
├── evaluate.py      # Test-set evaluation, per-class accuracy breakdown
├── probe.py         # Linear probing experiments (Section 4.3)
├── visualize.py     # Plotting: learning curves, accuracy heatmaps, attention
maps
└── run_sweep.py     # Launch the experimental grid
```

5.4 Key libraries

- **PyTorch** (`torch`, `torch.nn`): model, training
- **NumPy**: data generation, numerical work
- **Matplotlib**: plots for the report
- **Optionally** `wandb` or `tensorboard` for experiment tracking (nice to have, not essential)

5.5 Compute

Each individual experiment (one base, one p) trains a small model (~1–5M parameters) on short sequences. On a single modern GPU (even a T4 or V100), training to convergence should take 10–60 minutes. The full 120-experiment grid could run overnight. If you're using Duke's SLURM cluster, you can parallelize across GPUs.

---

6. What to Put in the Report (~4 pages)

1. **Introduction** (~0.5 page): The mystery from the Collatz paper, your simplified formulation, why it matters.
2. **Setup** (~0.5 page): Task definition, model architecture, training details. Keep it concise.
3. **Results** (~1.5 pages):
    - Accuracy heatmap (B vs. p) — this is the central figure.
    - Learning curves for select bases showing step-like behavior (or lack thereof).
    - Probing results: can you read off binary bits from hidden states?
    - Ablation: output-in-base-2 comparison.

4. **Analysis** (~1 page): Relate findings to $\nu_2(B)$, connect back to the Collatz paper's observations, discuss what the probing results tell us about internal representations.
5. **Conclusion** (~0.5 page): What did you learn? What remains open?

Key figures to include: (i) the accuracy heatmap, (ii) 2–3 learning curves, (iii) probe accuracy by layer, (iv) optionally an attention visualization.

---

7. Stretch Goals (if time permits)

- **Scaling laws:** How does accuracy change with model size (number of layers, embedding dim)? Can a deeper model learn higher p in odd bases?
- **Curriculum learning:** Does training first on small p and gradually increasing help odd-base models?
- **Alternative architectures:** Compare a standard transformer to a model with explicit positional structure (e.g., relative positional encodings, or feeding digit positions as features).
- **Connection to the full Collatz task:** If your residue-prediction model works well for some base B, can you use it as a "module" (frozen encoder) inside a model that predicts the long Collatz step?

---

8. References

- Charton & Narayanan, "Transformers know more than they can tell: Learning the Collatz sequence," arXiv:2511.10811 (2025).
- Nanda et al., "Progress measures for grokking via mechanistic interpretability," arXiv:2301.05217 (2023).
- Saxena et al., "Making hard problems easier with custom data distributions and loss regularization," arXiv:2410.03569 (2025).
- Vaswani et al., "Attention is all you need," NeurIPS 2017.
- Nogueira et al., "Investigating the limitations of transformers with simple arithmetic tasks," arXiv:2102.13019 (2021).